```
#include <list>
#include <mutex>
#include <algorithm>

std::list<int> some_list;                                    <-(1)
std::mutex some_mutex;                                       <-(2)

void add_to_list(int new_value)
{
    std::lock_guard<std::mutex> guard(some_mutex);           <-(3)
    some_list.push_back(new_value);
}
bool list_contains(int value_to_find)
{
    std::lock_guard<std::mutex> guard(some_mutex);                    (4)
    return std::find(some_list.begin(),some_list.end(),value_to_find)
        != some_list.end();
}
```

In listing 3.1, there's a single global variable (1), and it's protected with a corresponding global instance of `std::mutex` (2). The use of `std::lock_guard<std::mutex>` in `add_to_list()` (3) and again in `list_contains()` (4) means that the accesses in these functions are mutually exclusive: `list_contains()` will never see the list partway through a modification by `add_to_list()`.

Although there are occasions where this use of global variables is appropriate, in the majority of cases it's common to group the mutex and the protected data together in a class rather than use global variables. This is a standard application of object-oriented design rules: by putting them in a class, you're clearly marking them as related, and you can encapsulate the functionality and enforce the protection. In this case, the functions `add_to_list` and `list_contains` would become member functions of the class, and the mutex and protected data would both become `private` members of the class, making it much easier to identify which code has access to the data and thus which code needs to lock the mutex. If all the member functions of the class lock the mutex before accessing any other data members and unlock it when done, the data is nicely protected from all comers.